

A Working Handbook for FedLAW

Byzantine-Robust Federated Learning with Learnable Aggregation Weights

Concepts, the actual equations, and what each one means

Base paper: Parsa, Daghestani, Teixeira & Johansson, ICLR 2026 — arXiv:2511.03529

Equations transcribed from the paper; intuition and caveats added for study. Verify against the PDF before quoting in your thesis.

How to use this

This is not a replacement for reading the paper — it is the thing you keep open *while* you read it, so the symbols mean something. Each core equation appears exactly as the paper states it, followed by a plain-English “what this is doing” note. Three boxes recur: **grey** = the maths, **green** = the projection / mechanism detail, **amber** = a caveat that matters for *your* implementation specifically. The reading list is at the end.

1 The one idea

Every robust-FL method before FedLAW does two separate things: (1) *filter* suspected-malicious clients out, then (2) *average* whoever survives with equal weights. FedLAW collapses both into a single learnable object. It treats the aggregation weight on each client as a **trainable parameter**, optimised *jointly with the model itself*. Malicious clients are not filtered by a hand-coded rule — they are driven to (near-)zero weight by the optimisation, while honest-but-heterogeneous clients keep useful, non-uniform weights.

The objective (paper Eq. 6)

$$\min_{\theta \in \mathbb{R}^d, w \in \Delta_{t, \ell_0}^+} \sum_{i=1}^n w_i f_i(\theta)$$

$f_i(\theta)$ is client i 's local loss, θ is the shared model, $w = [w_1, \dots, w_n]$ are the aggregation weights. You are minimising a *weighted* sum of client losses over *both* the model and the weights at once. The whole method is: how do you solve this when some f_i belong to attackers, and what set Δ_{t, ℓ_0}^+ do you constrain w to?

2 The constraint set — where the robustness actually lives

The robustness is not in a clever filter; it is baked into the *set* that w is allowed to live in. This is the most important single object in the paper.

The sparse unit-capped simplex (paper Eq. defining Δ_{t, ℓ_0}^+)

$$\Delta_{t, \ell_0}^+ = \left\{ w \in \mathbb{R}^n \mid \sum_{i=1}^n w_i = 1, w_i \geq 0, w_i \leq t, \|w\|_0 \leq s \right\}$$

Four conditions, each doing a job:

- $\sum w_i = 1$ and $w_i \geq 0$ — it's a **probability distribution** over clients (a simplex). Weights are shares of trust.
- $w_i \leq t$ — the **cap**: no single client can dominate, even an honest one. Stops one client hijacking the model.

- $\|w\|_0 \leq s$ — the **sparsity** (ℓ_0 counts non-zeros): at most s clients get non-zero weight. *This is the Byzantine exclusion mechanism* — it forces up to $n - s$ clients (hopefully the attackers) to exactly zero.

The exact-exclusion special case (why your weights hit *exactly* zero)

The paper notes: if you set

$$t = \frac{1}{n - b_f}, \quad s = n - b_f,$$

where b_f is the number of Byzantine clients, then *every* feasible w has exactly $n - b_f$ non-zero entries, each equal to $1/(n - b_f)$, and the remaining b_f entries are zero. In words: with those settings the cap and sparsity together force a *uniform weight over exactly the surviving clients*.

Your validation reported malicious weights at *exactly* 0.000 from round 10. The paper’s Figure 1 shows malicious weights converging to *near-zero*, not identically zero. These are not the same claim, and the difference is the t, s setting above. If your t and s are set to the exact-exclusion values ($t = 1/(n - b_f)$, $s = n - b_f$), then exactly-zero is *expected* and correct — but it means the cap is doing the work and every surviving client is forced to identical weight, so you are *not* seeing the “adaptive non-uniform benign weights” that Figure 1 advertises. If you want to reproduce the paper’s qualitative figure (benign clients at *different* non-trivial weights), you need $s > n - b_f$ so there’s slack for the optimisation to actually choose weights. Worth checking which regime your config is in — it changes what your plots mean.

3 Solving it: alternating minimisation

You can’t minimise over θ and w together easily (it’s non-convex and they’re coupled). So FedLAW alternates: fix w , step θ ; then fix the new θ , step w . Two steps per global epoch.

3.1 Step A — update the model θ (given weights w)

Approximate each local loss by a quadratic around the current model θ_k :

Quadratic surrogate (paper)

$$\hat{f}_i(\theta; \theta_k) = f_i(\theta_k) + \langle \nabla_{\theta} f_i(\theta_k), \theta - \theta_k \rangle + \frac{1}{2\alpha} \|\theta - \theta_k\|_2^2$$

This is just a first-order Taylor expansion plus a quadratic “trust-region” term with radius controlled by α . Minimising $\sum_i w_i \hat{f}_i$ over θ gives a clean closed form:

$$\theta_{k+1}(w) = \theta_k - \alpha \sum_{i=1}^n w_i \nabla f_i(\theta_k) = \theta_k - \alpha G_k w$$

where $G_k = [\nabla f_1(\theta_k), \dots, \nabla f_n(\theta_k)]$ stacks the client gradients as columns.

What this is doing: it’s weighted gradient descent. The model step is the w -weighted average of client gradients. If a malicious client has $w_i \approx 0$, its gradient simply doesn’t enter the step. α is your model learning rate.

3.2 Step B — update the weights w (given the new model)

This is the hard, novel part. You want the weights that minimise the objective *taking into account that the weights also moved the model*:

Weight subproblem (paper)

$$w_{k+1} = \operatorname{argmin}_{w \in \Delta_{t, \ell_0}^+} \sum_{i=1}^n w_i f_i(\theta_k - \alpha G_k w)$$

Call this objective $\Phi_k(w)$. It's non-convex in w (the weights appear inside f_i via the model). FedLAW makes a quadratic surrogate $\hat{\Phi}_k$ around w_k , which turns the update into a **proximal mapping** — “take a gradient step, then project back onto the feasible set”:

$$w_{k+1} = \operatorname{argmin}_w \frac{1}{2} \|w - h_k\|_2^2 + \delta_{\Delta_{t, \ell_0}^+}(w) = \operatorname{prox}_{\Delta_{t, \ell_0}^+}(h_k)$$

where $\delta_{\Delta_{t, \ell_0}^+}$ is the indicator (0 inside the set, $+\infty$ outside), and h_k is the pre-projection gradient step.

3.3 The weight-update vector h_k — the equation you keep seeing

The gradient of Φ_k is approximated, and the result is the single most important formula to internalise:

The h_k update (paper)

$$h_k = w_k + \underbrace{\alpha \beta G_k^\top \tilde{G}_{k+1} w_k}_{\text{alignment reward}} - \underbrace{\beta \tilde{f}_{k+1}}_{\text{loss penalty}}$$

where:

- $\tilde{\theta}_{k+1} = \theta_k - \alpha G_k w_k$ is a **tentative** model (the model you'd get with current weights),
- $\tilde{G}_{k+1}$ = client gradients evaluated at $\tilde{\theta}_{k+1}$ (the second communication round),
- $\tilde{f}_{k+1}$ = client losses evaluated at $\tilde{\theta}_{k+1}$,
- β = the weight learning-rate / step size.

What the two terms do — this is the heart of the whole method:

- **Alignment term** $\alpha \beta G_k^\top \tilde{G}_{k+1} w_k$: the matrix $G_k^\top \tilde{G}_{k+1}$ is full of inner products between client gradients. A client whose gradient *agrees* with the consensus direction gets a *positive* contribution (weight goes up); a client pointing the opposite way (a sign-flipper) gets a *negative* one (weight goes down). This is the “do you point the same way as everyone else” signal.
- **Loss term** $-\beta \tilde{f}_{k+1}$: clients with *high* loss at the tentative model get their weight pushed down. Honest clients that are actually learning have lower loss.

A client must look good on *both* signals to keep weight. That “two independent tests” design is exactly why FedLAW beats methods that use only gradients (Krum) or only one round of information.

4 The projection (Algorithm 3) — where bugs hide

h_k is a raw vector; it won't satisfy the constraints (entries can be negative, won't sum to 1, too many non-zeros). The projection $\operatorname{prox}_{\Delta_{t, \ell_0}^+}(h_k)$ fixes that in three steps.

Three-step projection onto Δ_{t,ℓ_0}^+ (paper Algorithm 3):

1. **Select** the s largest entries of h_k .
2. **Take their support** (set all other entries to exactly zero — this enforces $\|w\|_0 \leq s$).
3. **Project those s entries** onto the *unit-capped simplex* $\{w \geq 0, \sum w = 1, w_i \leq t\}$ — a KKT/bisection routine (Wang & Lu 2015 style).

Complexity: $O(n \min(s, \log n) + s^2)$ for the projection; $O(dn)$ server memory for the weight update.

This routine is the single most error-prone piece of the implementation, because a wrong projection *still produces a plausible-looking weight vector* — it just silently breaks robustness. Your unit tests (sum-to-1, all entries in $[0, t]$, $\leq s$ non-zeros, known small cases) are exactly right and non-negotiable. If FedLAW ever “almost works”, suspect the capped-simplex step first: the cap t interacts with the simplex constraint in a way that’s easy to get subtly wrong (e.g. when several entries want to exceed the cap simultaneously).

5 The two communication rounds (Algorithm 2)

Most FL methods need one round per epoch (send model, get updates). FedLAW needs **two**, because the weight update requires gradients/losses at the *tentative* model $\tilde{\theta}_{k+1}$:

1. **Round 1:** broadcast θ_k . Clients return gradients $\nabla f_i(\theta_k)$ (and loss $f_i(\theta_k)$). Server forms G_k and the tentative model $\tilde{\theta}_{k+1} = \theta_k - \alpha G_k w_k$.
2. **Round 2:** broadcast $\tilde{\theta}_{k+1}$. Clients return gradients $\tilde{G}_{k+1}$ and losses $\tilde{f}_{k+1}$ at that point. Server computes h_k , projects to get w_{k+1} .
3. **Finalise:** update the real model $\theta_{k+1} = \theta_k - \alpha G_k w_{k+1}$ using the *new* weights and the *round-1* gradients.

Two implementation traps your project has already brushed against:

(a) **Loss timing.** The $\tilde{f}_{k+1}$ in h_k is the loss *at* $\tilde{\theta}_{k+1}$, before any local drift. In ByzFL, `compute_gradients()` gives you the loss at the current params (one batch); `compute_model_update()` returns the *mean loss during local training*, which has already moved. Use the former for the weight update — you’ve documented this, keep it.

(b) **Who supplies the Byzantine loss?** In the real method, a Byzantine client *submits* its loss — and a smart attacker can *forge a low loss* to neutralise the loss-penalty term (the “mimic” case the paper analyses in its resilience section). If your code *imputes* a benign placeholder loss (e.g. mean of honest losses) for Byzantine clients, you’ve quietly made the attacker weaker than the paper’s threat model, and your robustness will look better than it really is. This is the single most important thing to check before you trust any “it works” result.

6 The guarantees (what Theorems 2 and 3 actually say)

You do not need to reproduce the proofs, but you must be able to state what they promise — examiners ask.

Theorem 2 — Byzantine resilience (per step)

The aggregator is Byzantine-resilient with high probability: the bias η_k of the aggregated gradient relative to the true honest mean g is bounded. The bound decomposes into four error sources: *loss heterogeneity*, *gradient heterogeneity*, *inter-client variance*, and *mini-batch sampling noise*. Crucially, the analysis assumes a **gradient-clipping mechanism** on Byzantine updates (norm-bounding), *not* normalisation.

Theorem 3 — convergence (long run)

The alternating minimisation converges under attack. Weights w_k converge to a critical point w^* of the exact objective. For non-convex losses, the time-averaged squared gradient norm is bounded by $O(\zeta_\infty^2 + \sigma_{F,\infty}^2)$ (asymptotic bias and variance of the aggregator). The bias ζ_∞ is bounded by the *same* resilience quantity from Theorem 2 — so step-wise robustness and long-run convergence are formally linked.

Theorem 2’s resilience assumes Byzantine gradients are *norm-bounded by clipping*. One of the “fixes” floated for your α -calibration problem was to *unit-normalise* every client gradient before forming G_k . That changes the input geometry in a way the paper’s theory does not cover — you’d be deviating from FedLAW while claiming to extend it. The theory-preserving fix for the noise problem is *batch-averaging* (reduce mini-batch noise by averaging several batches), which changes estimate *quality*, not gradient *direction*. Prefer that.

7 The paper’s experimental setup (so you can match it)

For faithful comparison, mirror these:

- **Model:** a 3-layer MLP (*not* a CNN) on MNIST; also CIFAR-10.
- **Heterogeneity:** the paper uses a q -split (label-skew), $q \in \{0.6, 0.9\}$ — *higher q = more non-IID*. Note this is *not* Dirichlet- α ; if you use Dirichlet you must say so and not claim to reproduce their exact split.
- **Malicious fraction:** up to 40%. **Clients:** 200 (accuracy plots); 10 (weight-trajectory plots). **Batch:** 64. **Local epochs:** 3.
- **Attacks:** label-flipping, inverse-gradient, backdoor, double, LIE.
- **Baselines beaten:** Krum, Trimmed-Mean, Bulyan, Coordinate-wise Median, CCLIP, RFA, Huber.
- **Headline behaviour:** benign weights stabilise within ~ 20 rounds; malicious weights $\rightarrow$ near-zero.

The entire FedLAW setup has all 200 clients reporting every round. There is *no* partial-participation experiment. That absence is precisely the opening your dissertation targets: what happens to the learned weight w_i of a client that is *not sampled* this round? The paper gives no rule. That is the question you are answering.

8 Reading list — in order, with why

Read top-down. Do a 20-minute first pass (abstract, figures, conclusion) on each, then deep-read only the starred sections.

1. **FedLAW** — Parsa et al., arXiv:2511.03529. *Your base paper*. Deep-read ★Section 3 (the method) and ★the projection (Algorithm 3) until you can redraw Algorithm 2 from memory. This handbook maps to it section-by-section.
<https://arxiv.org/abs/2511.03529>
2. **FedAvg** — McMahan et al., arXiv:1602.05629. The base FL loop FedLAW modifies. Skim — you only need the averaging step and the local-epochs idea.
<https://arxiv.org/abs/1602.05629>
3. **Krum** — Blanchard et al., NeurIPS 2017. The classic distance-based defence and the *impossibility result* (no linear combination tolerates even one Byzantine worker) — this is *why* the field exists. Read the impossibility argument and the selection rule. (NeurIPS 2017; search “Machine Learning with Adversaries Krum”)
4. **Trimmed-Mean / Median** — Yin et al., arXiv:1803.01498. The statistical baselines, and the per-coordinate weakness under non-IID that motivates adaptive weighting.
<https://arxiv.org/abs/1803.01498>
5. **DeMoA** — arXiv:2509.02970. *Defines your gap and is your headline comparison*. It establishes partial participation as the open Byzantine-FL problem and solves it for *momentum* (not learnable weights). Read the problem framing and how it differs from what you’re doing.
<https://arxiv.org/abs/2509.02970>
6. **FedAU** — arXiv:2306.03401. The honest anchor for your direction: it does participation-aware weight adaptation and *explicitly* names “incorporate weight adaptation into other FL algorithms” as future work. Cite this.
<https://arxiv.org/abs/2306.03401>
7. **MIFA** — Gu et al., NeurIPS 2021. The original “reuse the last update from absent clients” idea your persistence mechanism adapts (so you cite it rather than claim to invent caching). (NeurIPS 2021; search “MIFA arbitrary device unavailability”)
8. **ByzFL** — arXiv:2505.24802. The library, not theory. Read alongside the code.
<https://arxiv.org/abs/2505.24802>
9. *Naming-collision note*: there is an *older* “FedLAW” (Li et al., ICML 2023, “Revisiting Weighted Aggregation”) that is *not* Byzantine and *did* test partial participation. You must cite and distinguish it so an examiner knows you know.

Equations transcribed from arXiv:2511.03529 and its public review; intuition, caveats, and reading notes added for study. Where the paper and your implementation disagree, the paper wins — and where this handbook and the paper disagree, the paper wins.